

9. Geometry

Contents

Geometry Notes	1
Geometry (X, Y, dot, cross)	3

Geometry Notes

```
1  Generate 2 points on a line
2  // ax + by + c = 0
3  if(a == 0){
4      p1 = {0, -1.0 * c/b};
5      p2 = {1, -1.0 * c/b};
6  }
7  else{
8      p1 = {-1.0 * c/a, 0};
9      p2 = {-1.0 * (c + b)/a, 1};
10 }
11
12
13 You're given 4 integers which are the
14 coefficients A B and C of the normal equation of the
15 straight line and a distance value R.
16
17 void solve() {
18     double a,b,c,r; cin >> a >> b >> c >> r;
19     double base = sqrt(a * a + b * b);
20     a /= base;
21     b /= base;
22     c /= base;
23     cout << setprecision(15) << a << ' ' << b << ' ' << c + r << endl;
24     cout << setprecision(15) << a << ' ' << b << ' ' << c - r << endl;
25 }
26
27 Area of the sector without an angle = (l * r) / 2
28 The length of the arc l = (theta / 360) * 2 * pi * r
29
30 The area of the parallelogram = the cross-product of
31 2 adjacent sides = 2 * area of the triangle made by 3 points.
32
33 Given 1 side of the Pythagorean Triangle ... Get the missing 2 sides :
34 ll n;
35 cin >> n;
36 if (n == 1 || n==2)
37     cout << -1;
38 else if (n & 1)
```

```

39     cout << (n * n + 1) / 2 << " " << (n * n - 1) / 2;
40 else
41     cout << n * n / 4 + 1 << " " << n * n / 4 - 1;
42
43 In triangle abc angle bac  $\cos(\theta) = (b^2 + c^2 - a^2) / (2 * b * c)$ 
44
45 n = number of sides of a regular polygon
46 S = side length of the polygon
47 ap = apothem the distance from the center of the polygon to the middle of any side
48 r = radius of the polygon which is the distance from the center of the polygon to any corner.
49 p = perimeter of the polygon
50
51 p = S * n
52 ap = S / (2 * tan(180/n)) = r * cos(180/n)
53 r = S / (2 * sin(180/n)) = ap / cos(180/n)
54 Area = (p * ap)/2 , (S^2 * n) / (4 * tan(180/n)) = ap^2 * n * tan(180/n)
55     = (r^2 * n * sin(360/n))/2
56
57 sin(2*theta) = 2 * sin(theta) * cos(theta)
58 cos(2*theta) = cos(theta)^2 - sin(theta)^2 = 2 * cos(theta)^2 - 1
59     = 1 - 2 * sin(2*theta)^2
60 sin(theta)^2 = (1 - cos(2 * theta))/2
61 cos(theta)^2 = (1 + cos(2 * theta))/2
62 tan(2*theta) = (2 * tan(theta)) / (1 - tan(theta)^2)
63
64 Circle intersection r1,r2,d where r1 >= r2
65 If d = r1 + r2 they touch from outside
66 If d = r1 - r2 they touch from inside
67 If r1 - r2 < d < r1 + r2 they intersect in two points
68
69 Plane equation ax + by + cz + d = 0
70 AB = (Bx-Ax,By-Ay,Bz-Az)
71 AC = (Cx-Ax,Cy-Ay,Cz-Az)
72 AB x AC = (a,b,c)
73 a = (By-Ay)*(Cz-Az)-(Cy-Ay)*(Bz-Az)
74 b = (Bz-Az)*(Cx-Ax)-(Cz-Az)*(Bx-Ax)
75 c = (Bx-Ax)*(Cy-Ay)-(Cx-Ax)*(By-Ay)
76 d = -(a*Ax+b*Ay+c*Az)
77
78 // Checks if four points lie in the same plane or not
79 bool samePlane(point a,point b,point c){
80     // a * (b x c) = volume = 0
81     return (a.dot(b.cross(c)) == 0);
82 }
83
84 void solve() {
85     vector<point>v(4);
86     for (int i = 0; i < 4; ++i) {
87         cin >> v[i].x >> v[i].y >> v[i].z;
88     }
89     for (int i = 0; i < 4; ++i) {
90         v[i].x -= v[3].x;
91         v[i].y -= v[3].y;
92         v[i].z -= v[3].z;
93     }
94     cout << (samePlane(v[0],v[1],v[2]) ? "YES":"NO") << endl;
95 }

```

Geometry (X, Y, dot, cross)

```

1  /*
2   conj(a) -> a.imag() *= -1
3   abs(point) distance between (0,0) to this point
4   norm(point) squared magnitude -> real2 + imag2
5   hypot(x, y) -> sqrt(x2 + y2)
6   arg(vector) angle between this vector and x-axis
7   clamp(a, l, r) == min(r, max(l, a))
8   polar(rho, theta) -> make vector with length rho and angle theta
9   internal angle = (n - 2) * 180 / n
10  number of diagonals n * (n - 3) / 2
11  Area(p) = internal_points_cnt + (boundary_points/2) - 1
12  boundary_point in vector = gcd(|x2-x1|, |y2-y1|) + 1
13  line have infinity point, segment have to end points
14  vector(x, y) perpendicular to vector(-y, x) and (y, -x)
15 */
16
17 using ll = int64_t;
18
19 using ld = double;
20 using pt = complex<ld>;
21
22 const ll INF = 7e18;
23 const ld EPS = 1e-9;
24 const ld PI = acos(-1);
25
26 #define X real()
27 #define Y imag()
28
29 #define dot(a, b) (conj(a) * (b)).X
30 #define cross(a, b) (conj(a) * (b)).Y
31
32 int sign(ld x) {
33     return (x > EPS) - (x < -EPS);
34 }
35
36 struct compX{
37     bool operator()(pt a, pt b) const {
38         return a.X != b.X ? a.X < b.X : a.Y < b.Y;
39     }
40 };
41 struct compY{
42     bool operator()(pt a, pt b) const {
43         return a.Y != b.Y ? a.Y < b.Y : a.X < b.X;
44     }
45 };
46
47 // ===== line, segment =====
48
49 // projection of pt p onto line ab
50 pt project(pt a, pt b, pt p) {
51     pt ab = b - a;
52     return a + ab * dot(p - a, ab) / norm(ab);
53 }
54
55 // works for any orientation

```

```

56 bool onSegment(pt a, pt b, pt p) {
57     return sign(cross(b - a, p - a)) == 0 &&
58         sign(dot(p - a, p - b)) <= 0;
59 }
60
61 // ccw: >0 left, <0 right, =0 collinear
62 int ccw(pt a, pt b, pt c) {
63     return sign(cross(b - a, c - a));
64 }
65
66 // works for any pts
67 ld distanceToLine(pt a, pt b, pt p) {
68     return fabsl(cross(b - a, p - a)) / abs(b - a);
69 }
70
71 // works for any line
72 ld distanceToLine(ld A, ld B, ld C, pt p) {
73     return fabsl(A*p.X + B*p.Y + C) / abs(pt(A, B));
74 }
75
76 // works for any pts
77 ld distanceToSegment(pt a, pt b, pt p) {
78     if (dot(b - a, p - a) < 0) return abs(p - a);
79     if (dot(a - b, p - b) < 0) return abs(p - b);
80     return distanceToLine(a, b, p);
81 }
82
83 // works for intersecting lines (not parallel)
84 pt lineIntersect(pt a, pt b, pt c, pt d) {
85     pt ab = b - a, cd = d - c;
86     return a + ab * (cross(c - a, cd) / cross(ab, cd));
87 }
88
89 // works for all segments (returns intersection pt if exists)
90 bool segmentsIntersect(pt a, pt b, pt c, pt d, pt &inter) {
91     int d1 = ccw(a, b, c), d2 = ccw(a, b, d);
92     int d3 = ccw(c, d, a), d4 = ccw(c, d, b);
93
94     if(d1 * d2 < 0 && d3 * d4 < 0)
95         return inter = lineIntersect(a, b, c, d), true;
96
97     if(d1 == 0 && onSegment(a, b, c)) return inter = c, true;
98     if(d2 == 0 && onSegment(a, b, d)) return inter = d, true;
99     if(d3 == 0 && onSegment(c, d, a)) return inter = a, true;
100    if(d4 == 0 && onSegment(c, d, b)) return inter = b, true;
101
102    return false;
103 }
104
105 // works for any triangle
106 ld triangleArea(pt a, pt b, pt c) {
107     return 0.5 * fabs(cross(b - a, c - a));
108 }
109
110 bool ptInTriangle(pt a, pt b, pt c, pt p) {
111     ld s1 = cross(b - a, p - a);

```

```

112     ld s2 = cross(c - b, p - b);
113     ld s3 = cross(a - c, p - c);
114     return (sign(s1) >= 0 && sign(s2) >= 0 && sign(s3) >= 0) ||
115           (sign(s1) <= 0 && sign(s2) <= 0 && sign(s3) <= 0);
116 }
117
118 // angle abc in radians
119 ld angle_abc(pt a, pt b, pt c) {
120     return acos(clamp<ld>(dot(a - b, c - b) / (abs(a - b) * abs(c - b)), -1, 1));
121 }
122
123 // ===== Circles =====
124
125 pair<ld, pt> findCircle(pt a, pt b, pt c) {
126     pt m1 = (a + b) / 2.0, m2 = (b + c) / 2.0;
127     pt ab = b - a, bc = c - b;
128     pt center = lineIntersect(m1, m1 + pt(-ab.Y, ab.X),
129                               m2, m2 + pt(-bc.Y, bc.X));
130     return {abs(center - a), center};
131 }
132
133 vector<pt> lineCircleIntersect(pt a, pt b, pt center, ld r) {
134     pt ab = b - a, ao = center - a;
135     pt proj = a + ab * dot(ao, ab) / norm(ab);
136     ld d = abs(proj - center);
137     if (d > r + EPS) return {};
138     if (abs(d - r) < EPS) return {proj};
139     ld h = (ld)sqrtl(r*r - d*d);
140     pt dir = ab / abs(ab);
141     return {proj + dir * h, proj - dir * h};
142 }
143
144 // in 0, 1, 2 pts
145 vector<pt> circleCircleIntersect(pt c1, ld r1, pt c2, ld r2) {
146     ld d = abs(c2 - c1);
147     if(d > r1 + r2 + EPS || d < abs(r1 - r2) - EPS) return {};
148     if(abs(d) < EPS && abs(r1 - r2) < EPS) return vector(3, c1); // infinity intersection
149
150     ld a = (r1*r1 - r2*r2 + d*d) / (2 * d), h2 = r1*r1 - a*a;
151     if (h2 < -EPS) return {};
152
153     pt dir = (c2 - c1) / d, p = c1 + dir * a;
154     if (abs(h2) < EPS) return {p};
155     ld h = sqrt(h2);
156     pt offset = dir * pt(0, 1) * h;
157     return {p + offset, p - offset};
158 }
159
160 pair<ld, pt> minimumEnclosingCircle(vector<pt> p) {
161     using circle = pair<ld, pt>;
162     shuffle(p.begin(), p.end(), mt19937(random_device{}()));
163     auto contains = [](circle c, const vector<pt>& pts) {
164         return all_of(pts.begin(), pts.end(),
165                       [&](auto p) {return abs(p - c.second) <= c.first + EPS;});
166     };
167     auto circleFrom2 = [](pt a, pt b) {

```

```

168         pt c = (a + b) / 2.0;
169         return circle{abs(a - c), c};
170     };
171     auto circleFrom3 = [](pt a, pt b, pt c) {
172         pt ab = (a + b) / 2.0, ac = (a + c) / 2.0;
173         pt ab_perp = (b - a) * pt(0, 1), ac_perp = (c - a) * pt(0, 1);
174         pt o = lineIntersect(ab, ab + ab_perp, ac, ac + ac_perp);
175         return circle{abs(o - a), o};
176     };
177     vector<pt> R;
178     function<circle(int)> welzl = [&](int n) -> circle {
179         if (n == 0 || R.size() == 3) {
180             if (R.empty()) return {};
181             if (R.size() == 1) return {0, R[0]};
182             if (R.size() == 2) return circleFrom2(R[0], R[1]);
183             return circleFrom3(R[0], R[1], R[2]);
184         }
185         pt q = p[n - 1];
186         circle D = welzl(n - 1);
187         if (contains(D, {q})) return D;
188         R.push_back(q);
189         auto res = welzl(n - 1);
190         R.pop_back();
191         return res;
192     };
193     return welzl((int)p.size());
194 }
195
196 // ===== polygon =====
197
198 // works for any polygon (returns +1 for ccw, -1 for cw)
199 ld polygonSign(vector<pt>& p) {
200     ld area = 0;
201     int n = (int)p.size();
202     p.push_back(p[0]);
203     for(int i = 0; i < n; ++i) area += cross(p[i], p[i + 1]);
204     p.pop_back();
205     return sign(0.5 * area);
206 }
207
208 // works for any polygon (removes dups, enforces ccw order)
209 void normPolygon(vector<pt>& p) {
210     vector<pt> res;
211     for(auto i : p) if(res.empty() || abs(i - res.back()) > EPS)
212         res.push_back(i);
213
214     if(res.size() > 1 && abs(res.front() - res.back()) < EPS)
215         res.pop_back();
216
217     if(polygonSign(res) < 0) reverse(res.begin(), res.end());
218
219     p = res;
220 }
221
222 // works for simple polygons with integer coordinates
223 ll internalPointsCount(vector<pt>& p) {

```

```

224     ll A2 = 0, B = 0;
225     int n = (int)p.size();
226     p.push_back(p[0]);
227     for (int i = 0; i < n; ++i) {
228         pt a = p[i], b = p[i + 1];
229         A2 += ll(a.X * b.Y - a.Y * b.X);
230         B += __gcd((ll)abs(b.X - a.X), (ll)abs(b.Y - a.Y));
231     }
232     p.pop_back();
233     return (abs(A2) - B + 2) / 2;
234 }
235
236 // works for any polygon (cw or ccw, convex or not)
237 ld polygonArea(const vector<pt>& p) {
238     int n = (int)p.size();
239     ld area = 0;
240     for (int i = 0; i+1 < n; ++i)
241         area += cross(p[i], p[i + 1]);
242     area += cross(p.back(), p.front());
243     return fabsl(area) / 2.0;
244 }
245
246 // works for any polygon (cw or ccw, convex or not)
247 bool ptInPolygon(const vector<pt> &p, pt o) {
248     int in = 0, n = (int)p.size();
249     for (int i = 0; i+1 < n; ++i) {
250         pt a = p[i], b = p[i + 1];
251         if (onSegment(a, b, o)) return true;
252         if (a.Y > o.Y != b.Y > o.Y) {
253             ld x = a.X + (b.X - a.X) *
254                 (o.Y - a.Y) / (b.Y - a.Y);
255             if(x > o.X) in ^= 1;
256         }
257     }
258     {
259         pt a = p.back(), b = p.front();
260         if (onSegment(a, b, o)) return true;
261         if ((a.Y > o.Y) != (b.Y > o.Y)) {
262             ld x = a.X + (b.X - a.X) *
263                 (o.Y - a.Y) / (b.Y - a.Y);
264             if(x > o.X) in ^= 1;
265         }
266     }
267     return in;
268 }
269
270
271 // work for simple convex polygon
272 bool ptInConvex(vector<pt> &poly, pt p) {
273     int n = int(poly.size());
274     if(n == 1) return sign(abs(poly[0] - p)) == 0;
275     if(n == 2) return onSegment(poly[0], poly[1], p);
276
277     pt f = poly[0];
278
279     if(sign(cross(poly[1] - f, p - f)) < 0 ||

```

```

280         sign(cross(poly[n - 1] - f, p - f)) > 0) return false;
281
282     int l = 1, r = n - 1;
283     while(r > l + 1) {
284         int mid = (l + r) >> 1;
285         if(sign(cross(poly[mid] - f, p - f)) > 0) l = mid;
286         else r = mid;
287     }
288     return ptInTriangle(f, poly[l], poly[r], p);
289 }
290
291 // works for any simple polygon (cw or ccw)
292 pt polygonCentroid(const vector<pt>& p) {
293     ld A = 0, c;
294     pt C(0, 0);
295     int n = (int)p.size();
296     pt cur, nxt;
297     for (int i = 0; i+1 < n; ++i) {
298         cur = p[i], nxt = p[i + 1];
299         c = cross(cur, nxt);
300         A += c;
301         C += (cur + nxt) * c;
302     }
303     cur = p.back(), nxt = p.front();
304     c = cross(cur, nxt);
305     A += c;
306     C += (cur+nxt) * c;
307
308     A *= 0.5;
309     if (abs(A) < EPS) return C;
310     return C / (6.0 * A);
311 }

```